LSGI3245 – Geospatial Database Management and Design

Lecture 5 - Intermediate SQL

Instructor: YU Yue
Department of Land Surveying and Geospatial Science,
The Hong Kong Polytechnic University

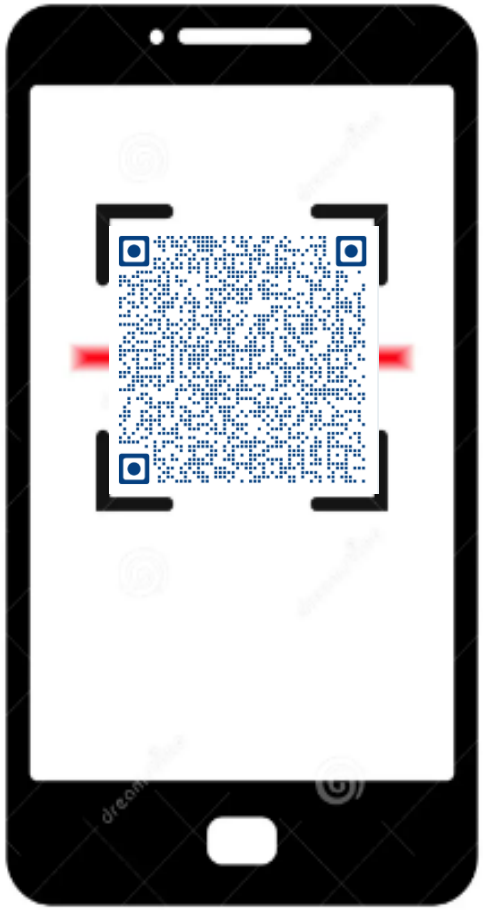
Tentative Schedule

Week	Lecture (Tuesday)		Lab (Wednesday)	
1	1-Sept	Course Introduction	Lab Introduction and Software Installation	2-Sept
2	8-Sept	Relational Model	Introduction to PostgreSQL (Part 1)	9-Sept
3	15-Sept	SQL (Part 1)	Introduction to PostgreSQL (Part 2)	16-Sept
4	22-Sept	SQL (Part 2) Release Information for Final Project	Query and Data Manipulation Using SQL (Part 1)	23-Sept
5	29-Sept	Intermediate SQL	Query and Data Manipulation Using SQL (Part 2)	30-Sept
6	6-Oct	Database Design	Introduction to PostGIS	7-Oct
7	13-Oct	Spatial Database & PostGIS	PostGIS: Spatial Datatype and Relationships	14-Oct
8	20-Oct	Test 1	PostGIS: Spatial Queries	21-Oct
9	27-Oct	PostGIS & Spatial Queries	Students Work on Final Projects (Part 1)	28-Oct
10	3-Nov	Spatial Storage and Indexing	Students Work on Final Projects (Part 2)	4-Nov
11	10-Nov	Test 2	Students Compile Final Project Reports (Part 1)	11-Nov
12	17-Nov	Project Presentation (Part 1)	Students Compile Final Project Reports (Part 2)	18-Nov
13	24-Nov	Project Presentation (Part 2)	Summary and Outlook	25-Nov

Final Project

Final Project Grouping

- 5 students per group (one as group leader)
  - Note: The assessment is based on the group as an integral part. The contributions of all the group members should be clearly stated in the oral presentation and the final written report, which are also considered for the final assessment of each group member.
- Give the group a good name
- Submit the list of group members via Google Spreadsheet
- **DDL**: by Sep 22, Tuesday (Week 4)
- If you have any queries about grouping, pls contact our TA for help: **Mr. Zhanpeng Zhang** (zhanpeng-2026.zhang@connect.polyu.hk) and **Mr. Feifan Lin** (Feifa.lin@polyu.edu.hk)

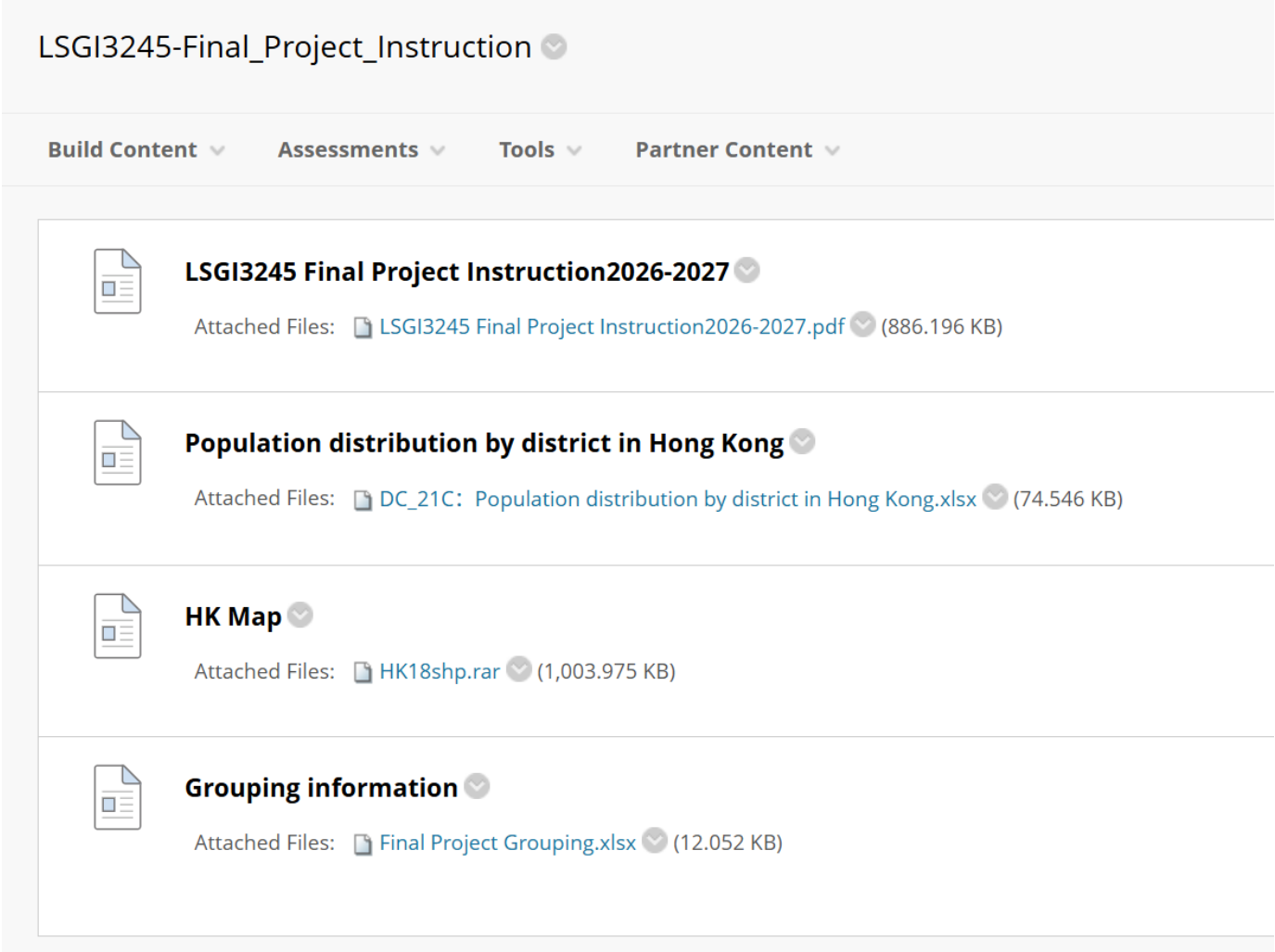


Please Scan the QR code to fill the group information

Overview of Final Project

- Weight: 15%
- Group: 5 students per group
- Deliverables: an oral presentation (10 min/group), a written report (with supplement SQL code)
- Task:
  - To incorporate both spatial and non-spatial information into a database design, and then implement them as a relational database; also demonstrate the mastery of spatial and non-spatial queries/operations over the database created.
- Instruction: Blackboard-Content/LSGI3245-Final_Project_Instruction

Final Project Instruction



Phase Test 1

Test Duration & Instructions



- Duration: **60 minutes** (***9:00 – 10:00 AM***)
- Instructions:
  - You will be provided with **a two-page exam paper** and **two blank answer sheets**.
  - Write down your **student ID and name** on BOTH the exam paper and the answer sheets.
  - **Answer ALL questions** from Section A and B. (total questions = 8, total marks = 100)
  - **Return the exam paper and answer sheets** to the instructor at the end of the exam.

Test Duration & Instructions



- Duration: **60 minutes** (***9:00 – 10:00 AM***)
- Format: **open-book** test

Warning: No Electronic Devices

Recap



- How to use nested **subqueries**?
  - *Subqueries in the where clause*
  - *Subqueries in the from clause*
  - *Subqueries in the with clause*

- Understand the usage of ***natural join***

- Know how to modify a database
  - ***delete***
  - ***insert***
  - ***update***

Learning Outcomes of Today's Class



- Understand different types of **join** expressions.

Join types
inner join
left outer join
right outer join
full outer join

Join conditions
natural
on <predicate>
using $(A_1, A_2, \ldots, A_n)$

- Understand the concept of ***view***.

Review of Key Concepts

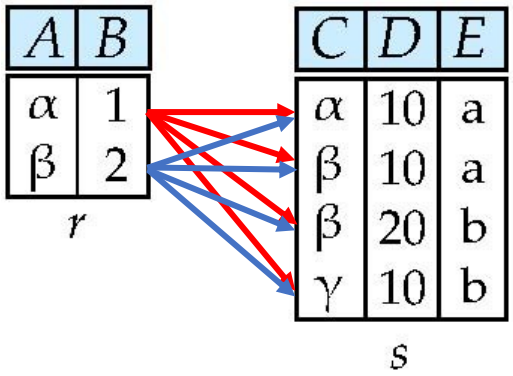
Relational Algebra Operators

Symbol (Name)	Example of Use
σ (Selection)	$\sigma_{salary >= 85000}(instructor)$
	Return rows of the input relation that satisfy the predicate.
Π (Projection)	$\Pi_{ID,\ salary}(instructor)$
	Output specified attributes from all rows of the input relation. Remove duplicate tuples from the output.
x (Cartesian Product)	*instructor* x *department*
	Output pairs of rows from the two input relations that have the same value on all attributes that have the same name.
∪ (Union)	$\Pi_{name}(instructor) \cup \Pi_{name}(student)$
	Output the union of tuples from the *two* input relations.
– (Set Difference)	$\Pi_{name}(instructor) - \Pi_{name}(student)$
	Output the set difference of tuples from the two input relations.
⋈ (Natural Join)	*instructor* ⋈ *department*
	Output pairs of rows from the two input relations that have the same value on all attributes that have the same name.

Join Two Relations – Cartesian Product

- The **Cartesian product** operation combines tuples from two relations, and its result contains all pairs of tuples from the two relations, regardless of whether their attribute values match.

Relation *r and s*:



A	B
α	1
β	2

r

C	D	E
α	10	a
β	10	a
β	20	b
γ	10	b

s

r x *s*:

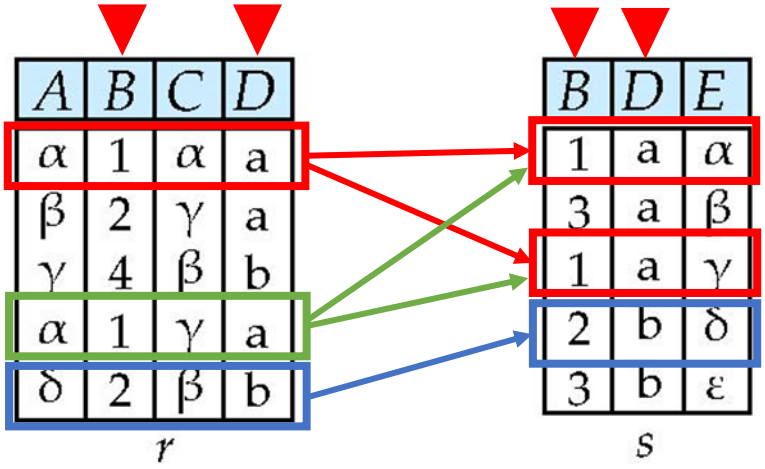
A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

```
select *
from r, s
```

Join Two Relations – Natural Join

- Different from the Cartesian product operation, **natural join** will give a result that contains the combination of tuples which have the same value on all attributes that have the same name.

Relations r, s:



A	B	C	D
α	1	α	a
β	2	γ	a
γ	4	β	b
α	1	γ	a
δ	2	β	b

r

B	D	E
1	a	α
3	a	β
1	a	γ
2	b	δ
3	b	ε

s

Natural Join

$r \bowtie s$

A	B	C	D	E
α	1	α	a	α
α	1	α	a	γ
α	1	γ	a	α
α	1	γ	a	γ
δ	2	β	b	δ

select *
from *r* **natural join** *s*

=

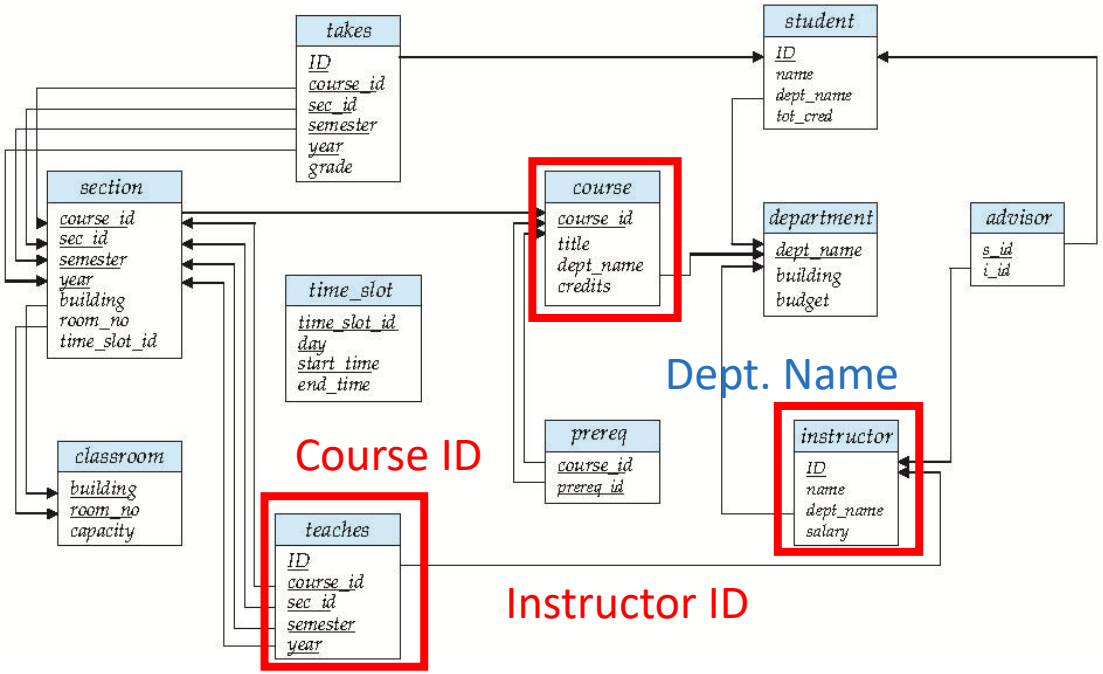
select *A, r.B, C, r.D, E*
from *r, s*
where *r.B* = *s.B* **and** *r.D* = *s.D*

Join Expressions

Group Discussion



List the names of **instructors** along with the titles of **courses** that they **teach** and offered by their own department.



select *name, title*
from *instructor* **natural join** *teaches*
natural join *course*

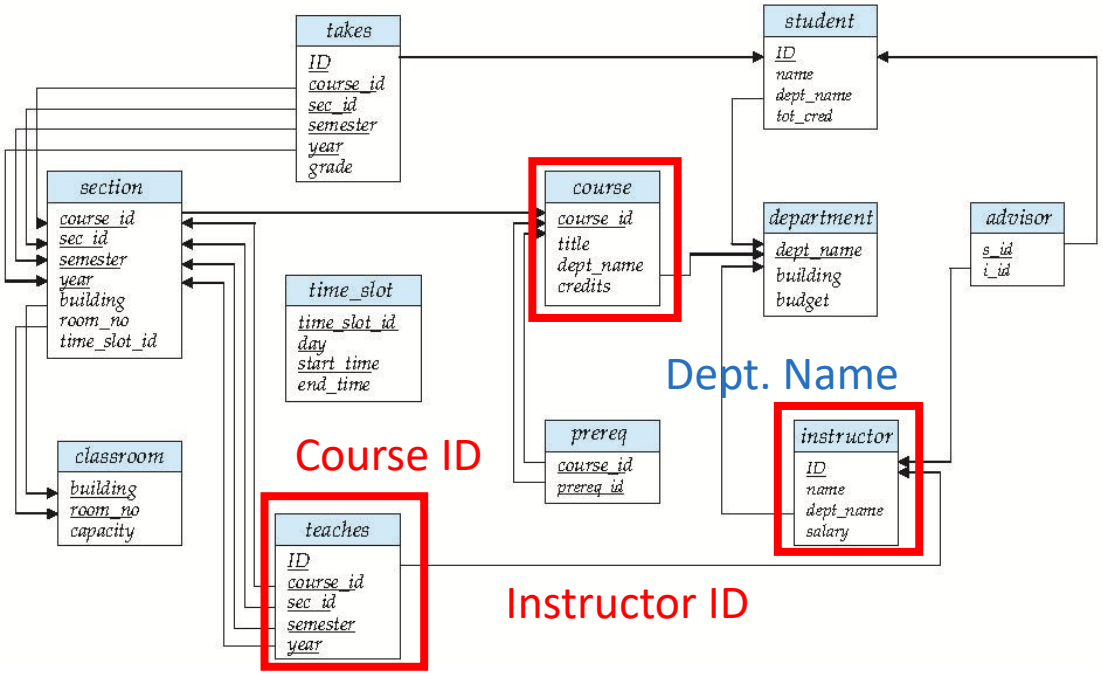
=

select *name, title*
from *instructor, teaches, course*
where *instructor.id=teaches.id* **and**
teaches.course_id=course.course_id **and**
instructor.dept_name=course.dept_name

Group Discussion



List the names of **instructors** along with the titles of **courses** that they **teach** ~~and offered by their own department~~.



What if we want to use join expression and circumvent **unrelated attributes** with the same name?

Solutions

List the names of **instructors** along with the titles of **courses** that they **teach** ~~and offered by their own department~~.

Cartesian Product + **where** <predicate>

select *name, title*
from (*instructor* **natural join** *teaches*), *course*
where *teaches.course_id=course.course_id*

=

select *name, title*
from *instructor, teaches, course*
where *instructor.id=teaches.id* **and**
teaches.course_id=course.course_id

r1 **join** r2 **using** (A1, A2)

select *name, title*
from (*instructor* **natural join** *teaches*) **join**
course **using** (*course_id*)

=

select *name, title*
from *instructor, teaches, course*
where *instructor.id=teaches.id* **and**
teaches.course_id=course.course_id

r1 **join** r2 **on** <predicate>

select *name, title*
from (*instructor* **natural join** *teaches*) **join**
course **on** *teaches.course_id=course.course_id*

=

select *name, title*
from *instructor, teaches, course*
where *instructor.id=teaches.id* **and**
teaches.course_id=course.course_id

Join Conditions

- The **on** condition allows a general predicate over the relations being joined.
- The **on** condition in this example specifies that a tuple from *(instructor* **natural join** *teaches)* matches a tuple from *course* if their *course_id* values are equal.
- The resulted relation has the *course_id* attribute listed twice.

r1 **join** r2 **on** <predicate>

select *name, title*
from (*instructor* **natural join** *teaches*) **join** *course* **on** *teaches.course_id=course.course_id*

=

select *name, title*
from *instructor, teaches, course*
where *instructor.id=teaches.id* **and** *teaches.course_id=course.course_id*

Join Expressions

- **Join operations** take two relations and return as a result of another relation.
- A join operation is a **Cartesian product** which requires that tuples in the two relations match **under some condition**. It also specifies the attributes that are present in the result of the join.
- The join operations are typically used as subquery expressions in the **from** clause.

Join types
inner join
left outer join
right outer join
full outer join

Join conditions
natural
on <predicate>
using $(A_1, A_2, \ldots, A_n)$

Outer Join

Outer Join

- An extension of the join operation that **avoids loss of information**.

course table

course_id	*title*	*dept_name*	*credits*
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

prereq table

course_id	*prereq_id*
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

select *
from *course* **natural join** *prereq*;

course_id	*title*	*dept_name*	*credits*	*prereq_id*
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101

Outer Join (Cont.)

- An extension of the join operation that **avoids loss of information**.

course table

course_id	*title*	*dept_name*	*credits*
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

prereq table

course_id	*prereq_id*
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

select *
from *course* **natural left outer join** *prereq*;

course_id	*title*	*dept_name*	*credits*	*prereq_id*
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	*null*

Outer Join (Cont.)

course table

course_id	*title*	*dept_name*	*credits*
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

prereq table

course_id	*prereq_id*
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

select *
from *course* **natural left outer join** *prereq*;

course_id	*title*	*dept_name*	*credits*	*prereq_id*
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	*null*

select *
from *course* **natural right outer join** *prereq*;

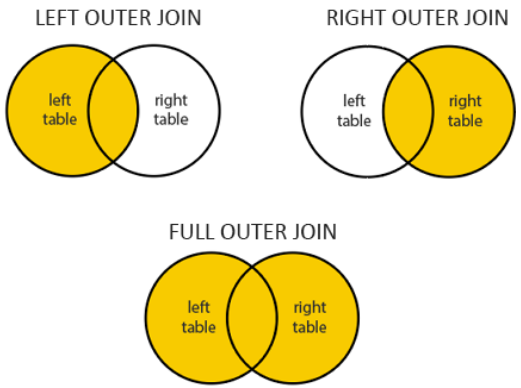
course_id	*title*	*dept_name*	*credits*	*prereq_id*
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-347	*null*	*null*	*null*	CS-101

select *
from *course* **natural full outer join** *prereq*;

course_id	*title*	*dept_name*	*credits*	*prereq_id*
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	*null*
CS-347	*null*	*null*	*null*	CS-101

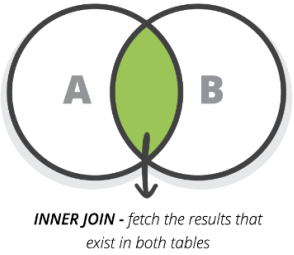
Outer Join (Cont.)

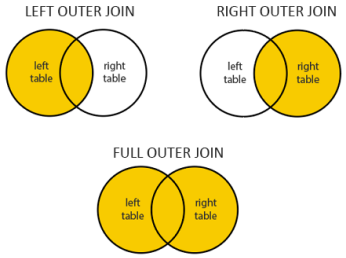
- An extension of the join operation that **avoids loss of information**.
- Computes the join and then adds tuples from one relation that **does not match** tuples in the other relation to the result of the join.
- Uses ***null*** values to fill non-matched attributes.
- Three forms of outer join:
  - left outer join
  - right outer join
  - full outer join



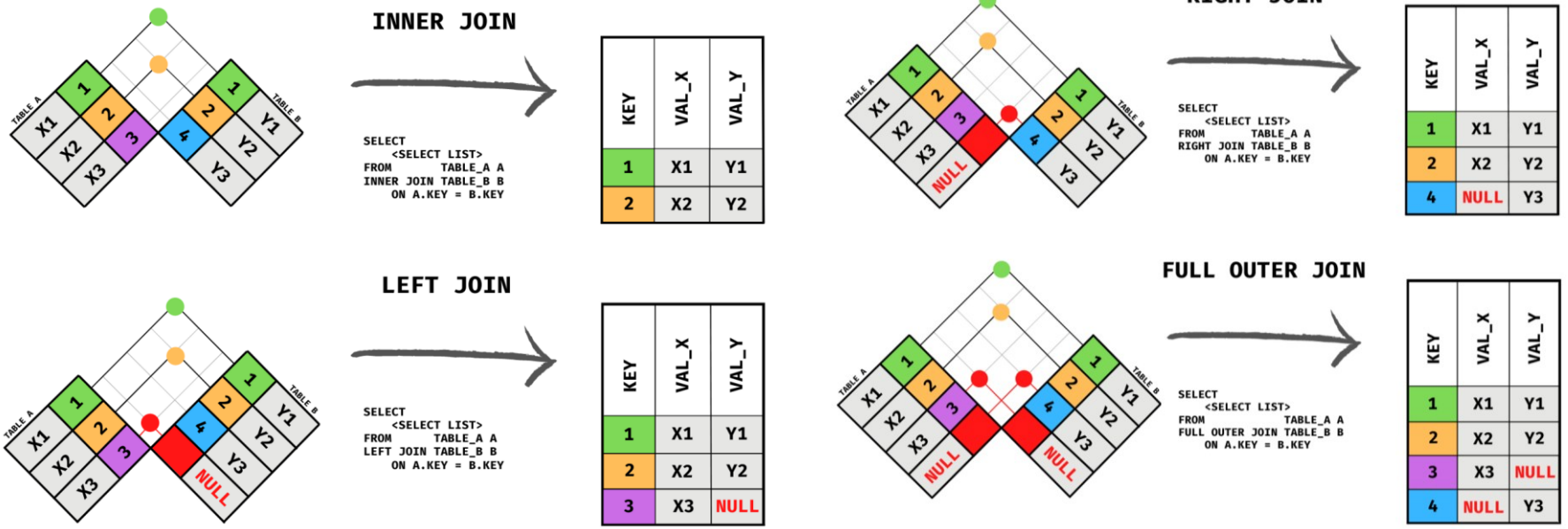
Source: Google Image

Outer Join (Cont.)





- **Inner join**: do not preserve non-matched tuples (when performing a SQL query, the keyword **inner** is optional)
- **Outer join**: preserve non-matched tuples by containing ***null*** values



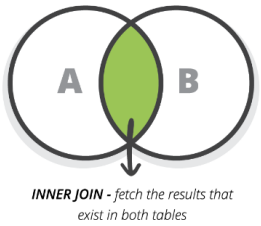
Summary of Join Types and Conditions

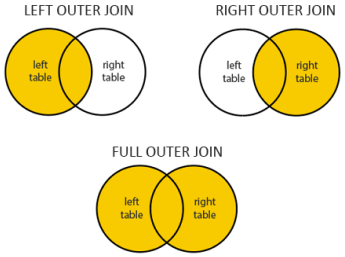
- **Join type** – defines how tuples in each relation that do not match any tuple in the other relation are treated.
- **Join condition** – defines which tuples in the two relations match, and what attributes are present in the result of the join.

Join types
inner join
left outer join
right outer join
full outer join

Any form of join can be combined with any join condition.

Join conditions
natural
on <predicate>
using $(A_1, A_2, \ldots, A_n)$





select * from

- r1 **natural join** r2
- r1 **join** r2 **on** r1.A1=r2.B1
- r1 **join** r2 **using** (A1)

Source: Google Image

Views

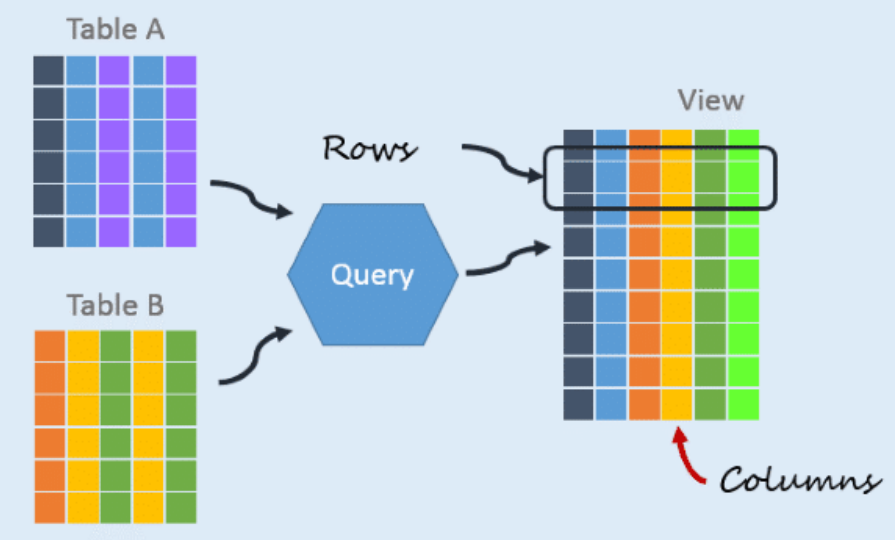
Views

- In some cases, it is not desirable for all users to see the entire logical model (that is, all the actual relations stored in the database.)
- Consider a person who needs to know an instructor's name and department, but not the salary. This person should see a relation described, in SQL, by

select *ID, name, dept_name*
from *instructor*

Views (Cont.)

- A **view** provides a mechanism to hide certain data from the view of certain users.
- Any relation that is not of the conceptual model but is made visible to a user as a "virtual relation" is called a **view**.



Source: Google Image

Views Definition

- A view is defined using the create view statement which has the form:

  create view v **as** <query expression>

  where <query expression> is any legal SQL expression. The view name is represented by *v.*

- Once a view is defined, the view name can be used to refer to the virtual relation that the view generates.
- View definition is not the same as creating a new relation by evaluating the query expression. Rather, a view definition causes the saving of an expression; the expression is substituted into queries using the view.

Example

- A view of instructors without their salary:

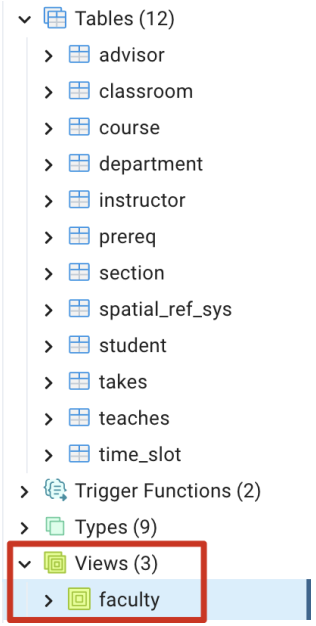
```
create view faculty as
    select ID, name, dept_name
    from instructor
```

Data Output | Explain | Messages | Query History

	id character varying (5)	name character varying (20)	dept_name character varying (20)
1	10101	Srinivasan	Comp. Sci.
2	12121	Wu	Finance
3	15151	Mozart	Music
4	22222	Einstein	Physics
5	32343	El Said	History
6	33456	Gold	Physics
7	45565	Katz	Comp. Sci.
8	58583	Califieri	History
9	76543	Singh	Finance
10	76766	Crick	Biology
11	83821	Brandt	Comp. Sci.
12	98345	Kim	Elec. Eng.

- Find all instructors in the Biology department:

```
select name
from faculty
where dept_name = 'Biology'
```



Materialized Views

- Certain database systems allow view relations to be physically stored.
  - Physical copy created when the view is defined. Such views are called materialized view
  - Materialized views in PostgreSQL:

PostgreSQL **create materialized view** v **as** <query expression>

- If relations used in the query are updated, the materialized view result becomes out of date
  - Need to maintain the view, by updating the view whenever the underlying relations are updated.

PostgreSQL **refresh materialized view** v

Integrity Constraints

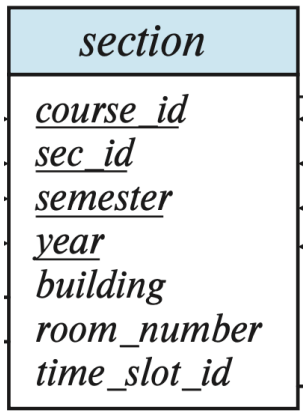
Integrity Constraints

- Integrity constraints ensure that changes made to the database by authorized users do not result in a loss of data consistency.
- Thus, integrity constraints guard against accidental damage to the database.
- Examples of integrity constraints:
  - An instructor name cannot be *null*.
  - No two instructions can have the same instructor *ID*.
  - Every department name in the *course* relation must have a matching department name in the department relation.
  - The budget of a department must be greater than $0.00.

Constraints on A Single Relation

- Example:
  - *building* and *room number* are not null
  - ensure that *semester* is one of the fall, winter, spring or summer.

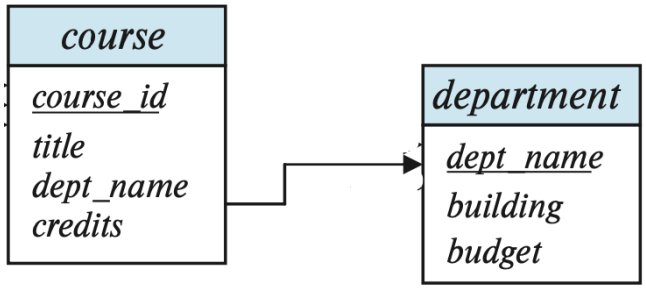
```
create table section (
    course_id varchar (8),
    sec_id varchar (8),
    semester varchar (6),
    year numeric (4,0),
    building varchar (15) not null,
    room_number varchar (7) not null,
    time slot id varchar (4),
    primary key (course_id, sec_id, semester, year),
    check (semester in ('Fall', 'Winter', 'Spring',
'Summer'))
);
```



Referential Integrity

- Often, we wish to ensure that a value that appears in one relation for a given set of attributes also appears for a certain set of attributes in another relation.
  - Example: If "Biology" is a department name appearing in one of the tuples in the *course* relation, then there exists a tuple in the *department* relation for "Biology".
- **Foreign keys** can be specified as part of the SQL **create table** statement:

```
create table course
    (course_id      varchar(8)
    title           varchar(50)
    dept_name       varchar(20)
    credits         numeric(2,0) check(credits > 0),
    primary key (course_id)
    foreign key (dept_name) references department)
```



Summary



- Understand different types of **join** expressions.

Join types
inner join
left outer join
right outer join
full outer join

Join conditions
natural
on <predicate>
using $(A_1, A_2, \ldots, A_n)$

- Understand the concept of ***view***.



Thank you!

michael-yue.yu@polyu.edu.hk

Source: Google Image